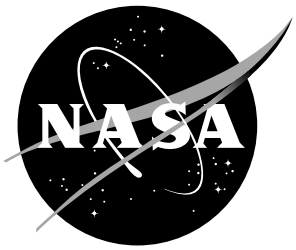
NASA/TM–20240013453



Design and Testing of an Octocopter for Aerodynamic and Power Consumption Modeling

Patrick E. Corrigan
Office of STEM Engagement (OSTEM) Internship Program, Hampton, Virginia

Justin J. Matt and Garrett D. Asper
Langley Research Center, Hampton, Virginia

March 2025

NASA STI Program Report Series

Since its founding, NASA has been dedicated to the advancement of aeronautics and space science. The NASA scientific and technical information (STI) program plays a key part in helping NASA maintain this important role.

The NASA STI program operates under the auspices of the Agency Chief Information Officer. It collects, organizes, provides for archiving, and disseminates NASA's STI. The NASA STI program provides access to the NTRS Registered and its public interface, the NASA Technical Reports Server, thus providing one of the largest collections of aeronautical and space science STI in the world. Results are published in both non-NASA channels and by NASA in the NASA STI Report Series, which includes the following report types:

- TECHNICAL PUBLICATION. Reports of completed research or a major significant phase of research that present the results of NASA Programs and include extensive data or theoretical analysis. Includes compilations of significant scientific and technical data and information deemed to be of continuing reference value. NASA counterpart of peer-reviewed formal professional papers but has less stringent limitations on manuscript length and extent of graphic presentations.
- TECHNICAL MEMORANDUM. Scientific and technical findings that are preliminary or of specialized interest, e.g., quick release reports, working papers, and bibliographies that contain minimal annotation. Does not contain extensive analysis.
- CONTRACTOR REPORT. Scientific and technical findings by NASA-sponsored contractors and grantees.
- CONFERENCE PUBLICATION. Collected papers from scientific and technical conferences, symposia, seminars, or other meetings sponsored or co-sponsored by NASA.
- SPECIAL PUBLICATION. Scientific, technical, or historical information from NASA programs, projects, and missions, often concerned with subjects having substantial public interest.
- TECHNICAL TRANSLATION. English-language translations of foreign scientific and technical material pertinent to NASA's mission.

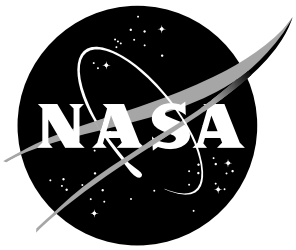
Specialized services also include organizing and publishing research results, distributing specialized research announcements and feeds, providing information desk and personal search support, and enabling data exchange services.

For more information about the NASA STI program, see the following:

- Access the NASA STI program home page at http://www.sti.nasa.gov
- Help desk contact information:

https://www.sti.nasa.gov/sti-contact-form/ and select the "General" help request type.

NASA/TM–20240013453



Design and Testing of an Octocopter for Aerodynamic and Power Consumption Modeling

Patrick E. Corrigan
Office of STEM Engagement (OSTEM) Internship Program, Hampton, Virginia

Justin J. Matt and Garrett D. Asper
Langley Research Center, Hampton, Virginia

National Aeronautics and
Space Administration

Langley Research Center
Hampton, Virginia 23681-2199

March 2025

Available from:

NASA STI Program / Mail Stop 148
NASA Langley Research Center
Hampton, VA 23681-2199
Fax: 757-864-6500

Abstract

This report presents the design, development, and testing of the Generic Aerodynamic Modeling for Multirotor Aircraft (GAMMA) vehicle, a multirotor aircraft designed to collect flight data for model identification. An avionics suite was selected based on findings from bench tests to ensure that motor speed and power consumption can be measured accurately in flight. A custom maneuver injection software was developed for GAMMA that creates the capability to perform automated flight test maneuvers. This software was implemented using the Mathworks® UAV Toolbox and maintains the original control system of the PX4 open-source autopilot, eliminating the need to develop custom control logic for this capability. Moreover, additional functionalities were added to the software to improve flight test efficiency and safety, such as the creation of parameters for configuring flight maneuvers and the implementation of custom failsafe logic. Preliminary flight test results are presented which demonstrate the effectiveness of the developed approach for collecting flight data for performance and flight dynamics modeling.

Contents

List of Figures

List of Tables

Nomenclature

ALIFT	Autonomy Lab for Intelligent Flight Technology
BEC	Battery Eliminator Circuit
CAN	Controller Area Network
CERTAIN	City Environment Range Testing for Autonomous Integrated Navigation
COTS	Commercial Off-the-Shelf
ESC	Electronic Speed Controller
eVTOL	Electric Vertical Takeoff and Landing
GAMMA	Generic Aerodynamic Modeling for Multirotor Aircraft
GCS	Ground Control Station
GCSO	Ground Control Station Operator
MAVROS	Micro Air Vehicle Robot Operating System
NED	North, East, Down
PDB	Power Distribution Board
PTI	Programmed Test Input
PWM	Pulse Width Modulation
QGC	QGroundControl
RC	Radio Control
RPIC	Remote Pilot in Command
RPM	Revolutions Per Minute
RTL	Return to Launch
SFC	Services, Functions, and Capabilities
UAM	Urban Air Mobility
UAS	Unmanned Aircraft System
UAV	Unmanned Aerial Vehicle
UDP	User Datagram Protocol
uORB	Micro Object Request Broker
USB	Universal Serial Bus

1 Introduction

NASA's System Wide Safety project aims to develop highly-assured methods to mitigate and reduce safety risks to future aviation operations, including the operation of unmanned aircraft systems (UAS). Within this project, a collection of Services, Functions, and Capabilities (SFCs) is in development to monitor and resolve unwanted precursors, anomalies, or trends observed during flight operations [1–3]. Multiple safety analysis capabilities in this framework require flight dynamics and performance models to predict and assess flight trajectories, power consumption, and operational envelope limits. To date, a Flight Performance Assessment Service (FPAS) has been developed to support this framework [3, 4]. FPAS employs a vast aerodynamic database collected from multiple experiments at the NASA Langley Research Center 12-Foot Low-Speed Tunnel to accurately simulate flight trajectories for multirotor aircraft based on a desired flight plan [5–7]. Present efforts aim to improve upon the accuracy and scope of FPAS by expanding simulation models to encompass multiple vehicle types, supporting real-time operation, modeling complex aerodynamic phenomena such as interaction effects between propellers [7], and collecting flight data to validate and improve upon the aerodynamic models.

In support of these efforts, this report details the design, development, and testing of the Generic Aerodynamic Modeling for Multirotor Aircraft (GAMMA) vehicle. GAMMA is a low-cost multirotor platform designed to facilitate quick and effective flight testing for system identification. The vehicle design includes an avionics suite that was selected after bench testing to ensure motor speed and power consumption can be measured accurately in flight, which is critical for the modeling needs of the System Wide Safety project. Custom software was developed to create the capability to inject programmed test inputs (PTIs) in-flight, which will be used to fly maneuvers for system identification. This capability was implemented using a novel approach leveraging the MathWorks® UAV Toolbox. This approach maintains the original PX4 autopilot control laws [8] while allowing for injection of PTIs to individual motors. This is different from some past works, which involved injecting reference commands before the controller [9, 10] or designing custom flight control logic with PTI injections to the motors commands output from the controller [11]. Preliminary flight testing results demonstrate the effectiveness of the developed vehicle and custom maneuver injection software for multirotor aerodynamic modeling research.

This report details the work completed during a 10-week Summer 2024 internship within the Flight Dynamics Branch at NASA Langley Research Center. The report is organized as follows. Section 2 provides an overview of the GAMMA flight test vehicle. Section 3 summarizes the bench testing and selection of electronic speed controllers (ESCs) and current sensors for the aircraft. The development, integration, and configuration of the maneuver injection software within the PX4 control architecture is discussed in Section 4. Section 5 presents preliminary flight test results, and future research directions and conclusions are discussed in Section 6.

2 GAMMA Platform

The GAMMA vehicle, shown in Figure 1, is a multirotor aircraft platform designed to collect flight data for system identification. The airframe is a commercial off-the-shelf (COTS) octocopter previously tested in the NASA Langley Research Center 12-Foot Low-Speed Tunnel [6]. The takeoff weight of the vehicle is approximately 18 pounds. At the center of the carbon fiber airframe is a two-story avionics bay that houses the flight computer and other instrumentation. Eight foldable aluminum arms, each measuring sixteen inches long, extend from the avionics bay. The vehicle is equipped with eight KDE4213-XF brushless DC electric motors which each drive a 15-inch propeller.

Figure 1: GAMMA vehicle.

GAMMA uses a Pixhawk® Cube Orange+ autopilot running PX4 open-source autopilot firmware [12]. The firmware was modified using the MathWorks UAV Toolbox [13] to support the injection of PTIs for system identification flight testing. In addition to the Cube, the full avionics suite includes the components listed in Table 1, which are used for command generation, communication, control, and data recording. A description and diagram of the wiring configuration is shown in Appendix A. The open-source ground control station (GCS) QGroundControl (QGC) is used for mission planning and vehicle software configuration.

Table 1: GAMMA avionics.

Component	Brand	Model
GPS	CubePilot	Here3 [14]
Flight Computer	CubePilot	Cube Orange+ [15]
BEC	Castle Creations	Pro 20A [16]
Radio Modem	RF Design	900x [17]
Remote Receiver	Spektrum	SPM9645 [18]
Logging Computer	Arduino	Mega 2560 [19]
Current Sensor	Pololu	ACS724 [20]
SD Card Reader for ACS724	Sparkfun Electronics	DEV-13743 [21]

3 Bench Testing and Avionics Selection

Because GAMMA is designed as a test bed for multirotor aerodynamic and power consumption modeling research, accurate measurements of current and RPM are essential. In pursuit of this goal, different hardware approaches to motor control, RPM sensing, and current sensing were investigated through bench testing, and the final avionics for the vehicle were selected based on findings from these tests. Two ESCs, the Castle Creations Phoenix Edge Lite 50A and Zubax Myxa B2, and a standalone ACS724 current sensor were evaluated. Table 2 highlights key differences between the two ESCs tested.

3.1 ESC Overview

The Phoenix Edge Lite ESC is equipped with a user-programmable auxiliary wire which can be used to record RPM data [22]. For bench testing, the auxiliary wire from the ESC was connected to a PJRC Teensy 4.1 Development Board, and the data was logged onto an SD card. Although the ESC features onboard logging capabilities, the maximum sample rate of 8 Hz is too low for many modeling applications, and the onboard current measurements proved inaccurate. Therefore, the ESC was connected to the Teensy board and a standalone ACS724 current sensor for RPM and current measurement recording. The wiring and logging setup is discussed further in Appendix B.2.

The Zubax Myxa B2 is based on the Telega motor control technology and communicates using the DroneCAN protocol [23], which enables logging of multiple signals onboard the flight computer, including RPM and current. In contrast to the Phoenix Edge Lite, which uses open-loop motor speed control, the Myxa B2 has several different motor control modes, including a closed-loop RPM control mode, which utilizes a feedback loop to drive the motor to the RPM commanded by the flight controller. The RPM control mode was employed during both bench and flight testing. The minimum and maximum RPM commands must be configured to ensure desired performance of the control loop. Because the maximum achievable motor speed is a function of supply voltage, the maximum RPM command was determined by measuring the RPM at full throttle in an open-loop control mode at the minimum flight voltage. This configuration ensures that the same maximum RPM can be achieved throughout the entire flight voltage range.

Table 2: Comparison of ESC properties.

	Phoenix Edge Lite	Zubax Myxa B2
Continuous Current Limit (Amps)	50	32
Weight (g)	56	55
Protocol	PWM	DroneCAN
Control Mode	Voltage	Voltage, Current, RPM
RPM Logging	Onboard, Aux Wire	CAN Bus
Current Logging	Onboard	CAN Bus

The bench test compared and evaluated data collection techniques for current and RPM measurements in order to select the avionics used for flight testing. An open-loop test signal, consisting of a sine wave followed by a series of step inputs, was commanded to an ESC using Simulink® to assess the system's performance under both static and dynamic loads. The ESC and motor were powered by a TDK-Lambda Z100-2-U programmable DC power supply [24]. This experiment was carried out once with each of the Myxa and Phoenix Edge Lite ESCs. Current measurements from the Myxa ESC and the Pololu ACS724 current sensor carrier board were compared to data from the power supply, which was recorded using a software interface [25]. RPM measurements from the Myxa and Phoenix Edge Lite were also recorded and are compared in the following section. The Myxa logs current and RPM measurements onboard the Cube through a CAN bus, whereas the external ACS724 current sensor and Phoenix Edge Lite RPM data are recorded on an external SD card connected to an Arduino Mega.

3.2 Current and RPM Data

Figure 2 compares current measurements from the Myxa ESC and the ACS724 sensor to measurements from the power supply. The Myxa ESC current measurements were calibrated using a linear transformation to align with the power source data. After calibration of the Myxa ESC, time-averaged current measurements from both sources were within approximately ±0.2 A of the power supply, which are expected to be sufficient for power consumption and performance modeling applications. Additionally, the data recorded onboard the Arduino Mega was accurately synchronized to the Cube Orange+ using the method detailed in Appendix B.1.

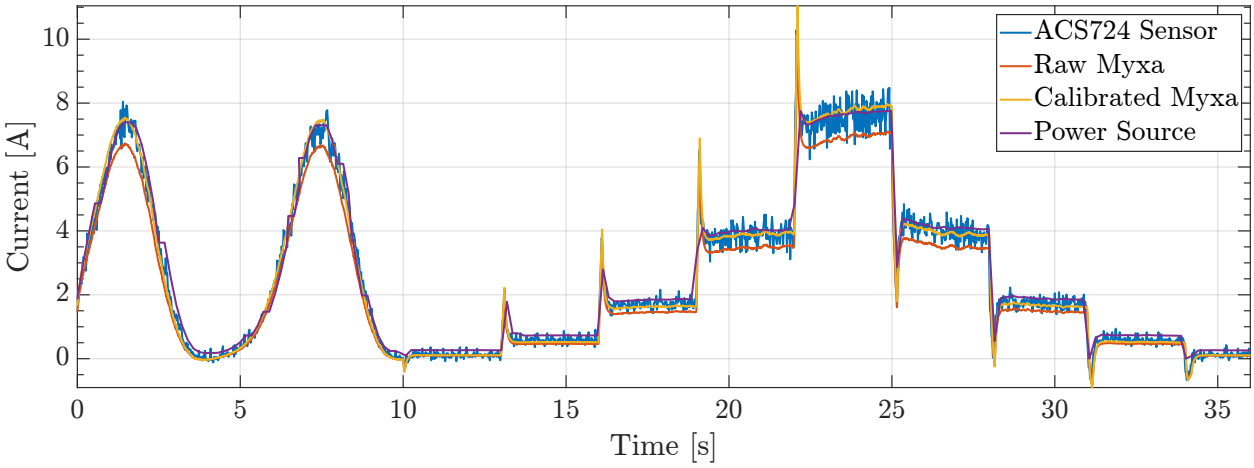


Figure 2: Current measurements from ESC bench test.

Figure 3 shows the RPM and throttle data recorded from both ESCs. Although the throttle input is identical for each ESC, the resulting motor speeds vary due to how PX4 handles the different control modes. Both measurements exhibit accurate RPM readings. However, a significant discrepancy is observed in the timing synchronization between the throttle input and measured RPM for the Phoenix Edge Lite ESC. As detailed in Appendix B.1, the RPM data from the Teensy was time synchronized with the Cube Orange+ data through an arming signal. The observed time delay is believed to be a result of the FreqMeasureMulti library [26] used on the Teensy for logging the analog RPM signal from the Phoenix ESC. Because the Myxa B2 provided simple integration with the Cube Orange+ and accurate current measurements, this ESC was ultimately selected for the vehicle, and the synchronization issue was not investigated further. Additionally, the vehicle was equipped with ACS724 current sensors in conjunction with the Myxa ESCs to provide two sources of current measurement that can be further evaluated from flight data.

4 Integration of Programmed Test Inputs with PX4

To support system identification flight testing, maneuver injection software was developed to create the capability to inject PTIs into the flight control system. This software capability was developed as a modification to the PX4 firmware using the MathWorks UAV Toolbox Support Package for PX4 Autopilots. The support package includes tools that enable integration between PX4 and Simulink, allowing users to embed algorithms on a Pixhawk flight computer through custom PX4 source code generation [27, 28]. Additionally, the toolbox allows users to read from and write to uORB topics, the publish-subscribe messaging system used by PX4 for inter-module communication, facilitating seamless control and data logging [29].

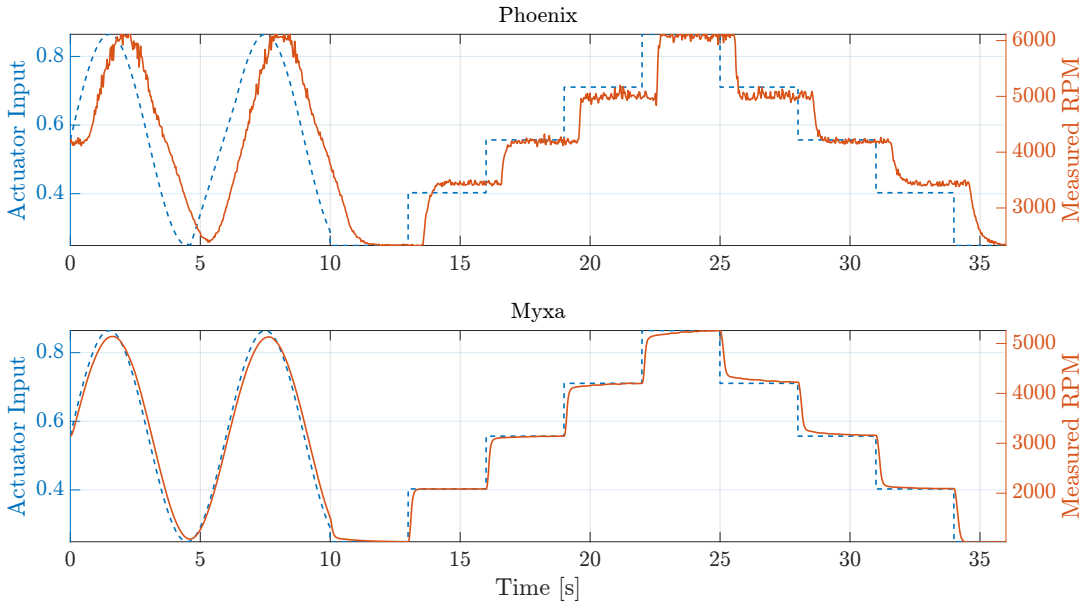


Figure 3: ESC bench test actuator input and measured RPM results.

One approach for implementing PTIs in UAS research has been through the use of a co-computer using the Micro Air Vehicle Robot Operating System (MAVROS) communication protocol [30] to interface with the MAVLink protocol [31] used by the flight computer. However, this method only allows for test signals to be injected upstream of the PX4 control algorithm and not at the actuator level [9, 10]. In Reference [11], the UAV Toolbox was used to develop software that implemented two types of PTIs: single-axis inputs summed with reference commands from the custom flight control algorithm for closed-loop system identification and multi-axis inputs summed with the motor command signals downstream of the control algorithm for bare-airframe system identification. This approach involved developing a custom control architecture to enable flight controls research in addition to system identification.

In contrast, the approach leveraged in this report implements the capability to inject PTIs at the motor throttle command while maintaining the standard PX4 control architecture. The UAV Toolbox was used to deploy the developed software. The PTIs are implemented by summing a test signal with the throttle command generated by the PX4 control system, as depicted in Figure 4. Injecting signals at this point is a common strategy for system identification flight testing for vehicles with active feedback control [32]. During PTI maneuvers, the control system can suppress the induced vehicle motion and create high correlation between inputs and feedback variables, making it difficult to determine individual control effectiveness and estimate model parameters. Summing the PTIs downstream of the control system ensures that each actuator is independently excited with a decorrelated test signal. Though the control system will distort the PTI signals and spoil properties such as orthogonality, this approach typically results in correlation between modeling variables that is low enough to accurately estimate model parameters [32].

In addition to implementing PTIs, several other enhancements were implemented using the UAV Toolbox to strengthen flight test efficiency and safety, including the creation of custom PX4 parameters [33] for refining PTI signal characteristics, the incorporation of custom logging topics for data analysis, and the implementation of failsafes to prevent the aircraft from losing stability during flight maneuvers and to ensure the vehicle remains within specified geofence boundaries. Moreover, a custom ground control station was designed using approaches developed in [33], enabling real-

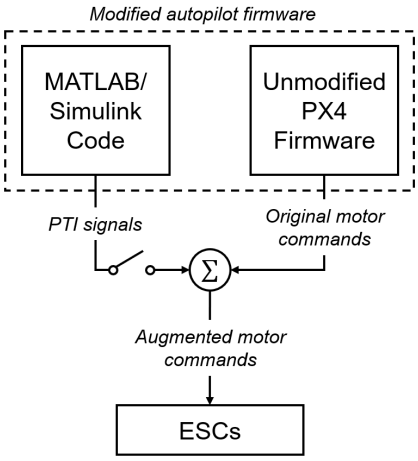


Figure 4: PTIs as implemented on GAMMA.

time monitoring of flight performance, PTI status, and geofence compliance. Figure 5 shows the high-level Simulink diagram designed to integrate these capabilities into a custom PX4 build. Collectively, these upgrades have led to efficient and reliable system identification flight testing for GAMMA by eliminating the need to develop custom control logic and by implementing effective safety assurance capabilities. The following subsections will detail individual components of the custom maneuver injection software. These include the interface with the radio control (RC) transmitter, the creation of custom PX4 parameters, the design and implementation of PTI signals, custom data logging capabilities, geofence functionality, and ground station capabilities.

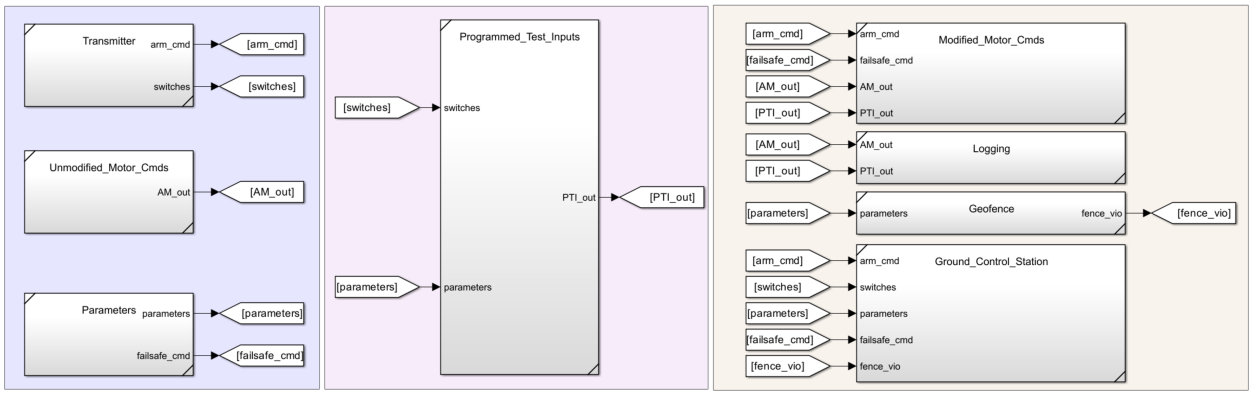


Figure 5: GAMMA custom software architecture in Simulink®.

4.1 Transmitter

A Spektrum iX20 RC transmitter and Spektrum DSMX remote receiver were used to relay piloted inputs to the Cube Orange+ flight computer. Table 3 details the functions of the ten channels used on the transmitter, which were based on Reference [11]. The first seven channels directly interface with the unmodified PX4 firmware and are configured through QGC. The remaining three channels are used to configure PTIs and are read into Simulink through the `input_rc` uORB message. Figure 6 illustrates the channel switch assignments utilized by the pilot, modeled after diagrams and functionality described in References [11, 34]. The three PTI switch channels control

the amplitude, mode, and arming status of the `Programmed_Test_Inputs` Simulink subsystem in Figure 5. The PTI amplitude knob provides a value ranging from 0 to 100%, indicating the amplitude percentage. The three-position PTI mode switch selects one of the following types of PTIs: single-motor doublets, mixer doublets, and multisines, which will be discussed further in Section 4.3. Once the mode is selected, the PTI arm switch adds the chosen PTI to the motor commands from PX4.

Table 3: GAMMA transmitter configuration (adapted from Reference [11]).

Channel	Label	Function	Setting	Switch Type
1	— —	Pilot input	Roll command	Right stick (↔)
2	— —	Pilot input	Pitch command	Right stick (↕)
3	— —	Pilot input	Throttle command	Left stick (↕)
4	— —	Pilot input	Yaw command	Left stick (↔)
5	B	Custom flight mode	Stabilized or Altitude	Three-position switch
6	C	Arm command	Armed or Disarmed	Three-position switch
7	Left Slider	Flight termination	Nominal or Terminated	Slider
8	G	PTI mode	Input type	Three-position switch
9	H	PTI arm	Off or On	Two-position switch
10	Right Knob	PTI amplitude	0 to 100%	Rotary knob

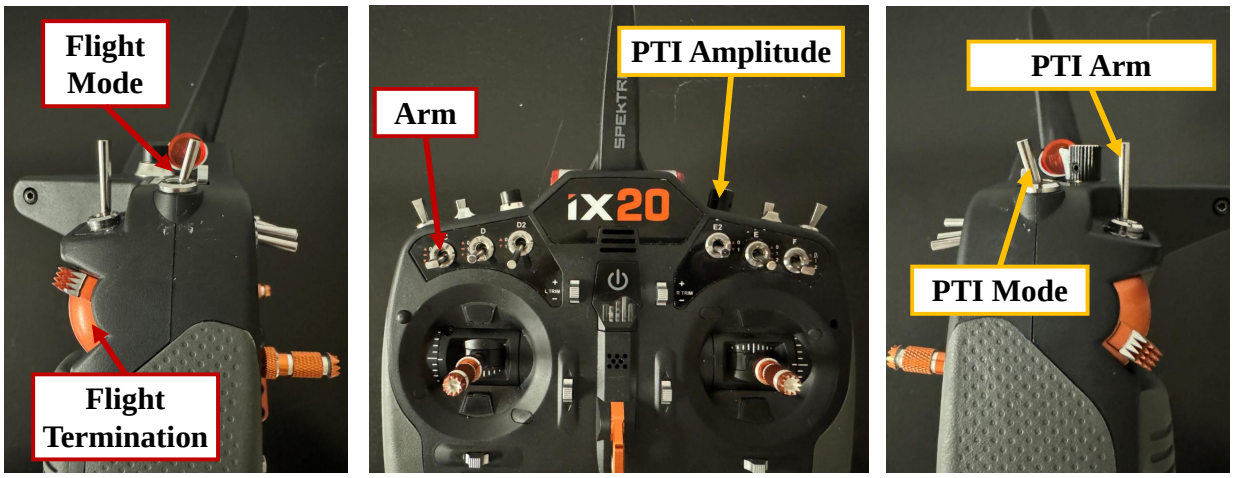


Figure 6: RC transmitter programmed for PTI injection flight testing (adapted from Reference [11]).

4.2 Parameters

Custom PX4 parameters were implemented to enable the GCS operator to configure PTIs during flight tests. This functionality allows parameters to be added to the QGC interface, referenced in Simulink, and updated in real time. To achieve this, the `module.yaml` file within the `battery_status` module in the PX4 base code was modified to define new parameters [33]. Table 4 provides an overview of each custom parameter added to the QGC interface for safe and

efficient PTI implementation. Both custom and standard QGC parameters can be read into a Simulink model using the PX4 Parameter Read block. The remainder of this section will reference these parameters and offer a more detailed description.

Table 4: QGC custom parameters.

Parameter	Short Description	Default Value
GAM_ROLL_LIM	Roll Limit during PTIs	45°
GAM_PITCH_LIM	Pitch Limit during PTIs	45°
GAM_PTI_ALTITUDE	Minimum Altitude required for PTIs	100 ft
GAM_FENCE_X_POS	Custom Geofence North Boundary	50 ft
GAM_FENCE_Y_POS	Custom Geofence East Boundary	50 ft
GAM_FENCE_Z	Custom Geofence Altitude Boundary	50 ft
GAM_FENCE_X_NEG	Custom Geofence South Boundary	50 ft
GAM_FENCE_Y_NEG	Custom Geofence West Boundary	50 ft
GAM_PTI_AMP	Maximum PTI Amplitude of transmitter knob	40%
GAM_DOUB_DUR	Total duration of both doublet PTIs	1 s
GAM_MS_T	Period of Multisine PTI	12.7 s

4.3 Programmed Test Inputs

4.3.1 Design

Three types of PTI signals were implemented into the Simulink diagram: orthogonal phase-optimized multisines, single motor doublets, and mixer doublets. Examples of each type of PTI are shown in Figure 7. It is worth noting that because the PTIs are summed with the output of the control system, the resultant motor commands will have a different shape than the PTIs shown. For clarity, a single multisine is shown, rather than all eight orthogonal multisines.

Orthogonal phase-optimized multisines are used to simultaneously excite each individual control actuator with test signals that are mutually orthogonal in both the time and frequency domain [32]. This type of PTI will be used to collect data for bare-airframe system identification. A multisine is a sum of sinusoids that contains discrete harmonic frequencies with specified power and can be expressed as a summation of sinusoids with unique, harmonically related frequencies [32]:

$$u(t) = \sum_{i=1}^{N_k} A_i \sin(2\pi k_i t/T + \phi_i) \tag{1}$$

where A_i are the amplitudes; ϕ_i are the phase angles; k_i are the harmonic numbers of each sinusoid component; T is the period of the multisine and, in this work, also the duration of the maneuver; and N_k is the number of harmonics in the multisine. The frequencies of each component are computed as $f_i = k_i/T$. Multisine phase angles can be optimized to minimize peak factor, which results in a low peak-to-peak response that is desirable for modeling [32]. This optimization was performed using the clipping algorithm discussed in Reference [35]. Multisine signals can be designed to be mutually orthogonal by selecting the harmonic frequencies, k, to be unique to each input. The specific characteristics of the multisines designed and demonstrated during flight tests are discussed further in Section 5.

Single-inputs maneuvers, such as doublet maneuvers, are often flown to collect data to validate the predictive capability of an identified model. For multirotor aircraft, a single motor doublet will

excite the vehicle about all degrees of freedom. In contrast, the mixer doublets were designed to create a PTI with a single-axis response. This results in a maneuver that is similar to doublets added at the attitude commands (manually by a pilot or automatically using a PTI), but allows for the same injection point and thus the same software framework to be used. The mixer doublets were implemented by passing the PTI signal through a mixer matrix, M, which allocates control to the eight motors based on the desired axis of motion.

$$M = \begin{bmatrix} -1 & 1 & -1 & -1 & 1 & 1 & 1 & -1 \\ 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 \\ -1 & -1 & 1 & 1 & 1 & 1 & -1 & -1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}^T$$

Commands are allocated to the motors according to the equation $u = Mv$, where u are the motor commands and v is a 4×1 vector of PTIs corresponding to roll, pitch, yaw, and heave motion.

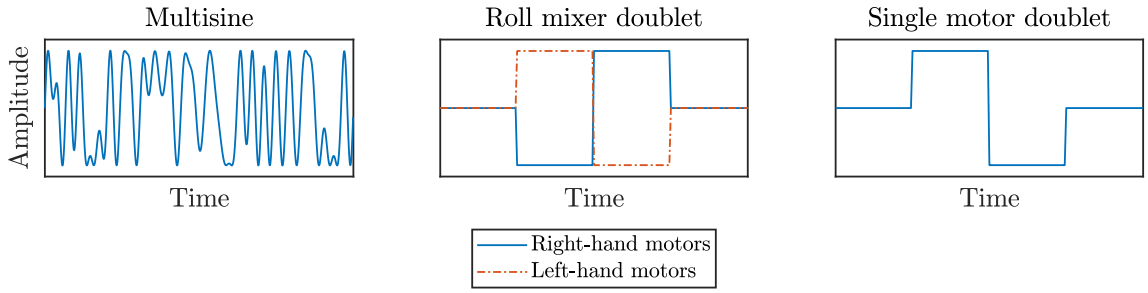


Figure 7: Example PTI types.

4.3.2 Implementation

PTIs were implemented using the UAV toolbox by reading motor commands from the PX4 controller, summing them with PTI signals, and sending the resulting commands to the ESCs, as shown in Figure 4. This approach reads the PX4 motor commands using the uORB topic `ActuatorMotors` and sends the augmented motor commands to the ESC using the PX4 PWM Output block provided in Simulink as part of UAV Toolbox. The PX4 PWM Output block is compatible with both CAN and PWM ESCs [36].

As described in Section 4.1, PTIs are selected and activated using two transmitter switches. The PTI arming switch initiates the maneuver, changing the PTI signal command from 0 to a value corresponding to the PTI type selected by the PTI mode switch. The PTI signal command adheres to safety protocols, ensuring the maneuver is only initiated above a specified altitude, controlled by the parameter GAM_PTI_ALTITUDE, and enforcing hard limits on the aircraft's roll and pitch angles via the parameters GAM_ROLL_LIM and GAM_PITCH_LIM. If any of these limits are exceeded during a maneuver, the PTI will automatically disable. The PTI signal command selects one of the three maneuvers shown in Figure 7.

The PTI signal is processed by a subsystem corresponding to the selected PTI mode, which initiates a timer and generates the normalized throttle signal, ranging from 0 to 1, for each ESC [11]. The subsystem outputs a single variable containing the scaled 8×1 vector of PTI excitation commands at the specified amplitude. Alongside the PTI signal command, various custom parameters are passed to each subsystem, enabling real-time configuration of the excitation signals from the ground station.

The orthogonal multisines PTI subsystem includes constant values for the phase angles of each sinusoidal component of the multisines. Several sets of phase angles that correspond to different multisine periods are stored in the code as lookup tables. This allows for the multisine period to be changed from the GCS using the custom parameter GAM_MS_T without significantly altering the frequency content, which is needed because maneuvers with different durations may be required for different flight conditions. For example, at higher ground speeds, shorter maneuvers may be needed due to airspace constraints. When the period is set via the GAM_MS_T parameter, the phase angles corresponding to the next-largest period in the table are used, and the frequencies are linearly scaled by the new period. This scaling ensures that the maximum frequencies of the multisines are always at or above the designed maximum frequency, which was selected at 3 Hz based on the predicted dynamics of the vehicle.

Two other subsystems implement the single-motor and mixer doublet PTIs. For both types of PTIs, maneuvers are automatically performed in series for each input (either individual motors or individual mixer channels). The duration of these doublets can be adjusted based on data collection requirements and spatial constraints by modifying the GAM_DOUB_DUR parameter, which controls the duration of the positive, negative, and neutral phases of each doublet.

The developed PTI approach improves flexibility and adaptability during system identification flight testing by enabling real-time adjustment of key parameters. Moreover, integrating PTI capabilities with the standard PX4 control system enhances the efficiency and safety of flight testing, as custom control logic does not need to be designed, programmed, and tested.

4.4 Logging Topics

The UAV Toolbox enables data from the Simulink model to be logged onboard the Cube Orange+ flight controller. Custom logging capabilities were created to record PTI signals and related data for modeling. As explained in Reference [37], after updating the SD card logging setup file on the Cube and configuring the custom logging topics in the PX4 directory, data can be written to the onboard SD card using the PX4 ULog block in Simulink. To effectively monitor the PTI excitations, two custom logging signals were implemented: `GammaPti` and `GammaOutput`. The `GammaPti` signal records the PTI signal value received from the `PTI_Implementation` subsystem. The `GammaOutput` signal logs the combined output of `GammaPti` and the signals from the `ActuatorMotors` uORB topic.

4.5 Geofence

For many UAS research facilities, incorporating a geofence is crucial to ensure the vehicle remains within a designated flight test area. QGC enables users to set multiple geofences for a mission by navigating to *Plan View* and selecting the *Fence* button [38]. Although multiple fences can be configured as polygons or circles, only one failsafe action can be programmed for geofence breaches. The GAMMA project aimed to implement a two-layer geofence system, preserving the unmodified PX4 geofence while adding an additional layer for enhanced flight safety. Figure 8 shows an example of the two geofences: the inner geofence triggers a Return to Launch (RTL) command using custom control logic via the Simulink diagram, and the outer geofence initiates a flight termination command through the unmodified PX4 firmware.

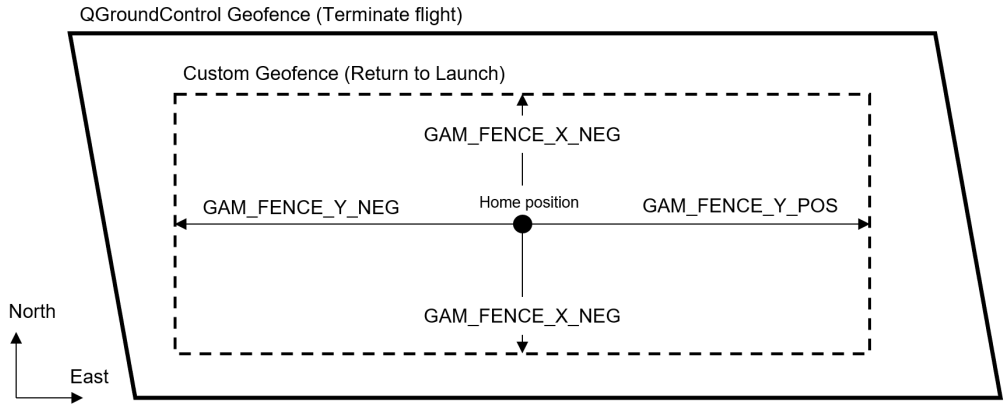


Figure 8: Example geofence boundaries.

For a circular geofence, the maximum horizontal distance parameter, GF_MAX_HOR_DIST, can be read into Simulink and a percentage of this radius or a fixed distance can be subtracted to define the inner geofence boundaries. Alternatively, for polygon geofences in QGC, as seen in Figure 8, the boundary points are not available in the vehicle's parameter list. Therefore, the inner geofence was instead defined using custom PX4 parameters that specify the maximum allowable distance the vehicle can travel in the North, East, and Down (NED) directions, which allowed for the creation of a custom rectangular geofence. All geofence parameters are depicted in Table 4.

The `VehicleLocalPosition` uORB topic provides the NED position of the vehicle, with the location where the vehicle was powered serving as the origin. When the inner geofence boundaries are breached, the vehicle is set to RTL mode by sending a uint32-type pulse of 20 to the `VehicleCommand` uORB topic [39]. This approach maintains the unmodified safety logic of the outer geofence while adding an extra layer of security with the RTL inner geofence, thereby increasing confidence in vehicle recovery in the event of a geofence breach.

4.6 Custom Ground Control Station

A custom GCS interface was developed in Simulink to allow the GCS operator (GCSO) to monitor key parameters and vehicle states in real time. The GCS was developed through collaboration with other UAS flight test projects at NASA Langley [11, 33]. This setup supports live monitoring of signals and parameters in both Simulink and QGC. As detailed in Reference [33], signals from the Simulink model can be transmitted to both QGC and custom GCS platforms by writing to the `DebugArray` uORB topic, which can be leveraged to send messages to the ground station. The `DebugArray` transmits information via MAVLink, a messaging protocol used by PX4 to communicate with ground stations [40]. By utilizing the UDP Receive block in Simulink, the `DebugArray` MAVLink message, along with other MAVLink data, can be monitored in real-time.

Figure 9 depicts the custom GCS used to monitor PTI settings and the custom geofence. The top section of the GCS interface displays the status of custom PTI parameters and transmitter switches, indicating mode changes, PTI type, amplitude, and arming status. The bottom section shows the vehicle's real-time position from the `VehicleLocalPosition` uORB topic relative to both the QGC geofence and custom geofence. The inner geofence violation status is determined by a boolean value from the GAMMA Simulink model, which is transmitted through the `DebugArray` uORB topic, while the outer geofence status is monitored via the `primary_geofence_breached` element of the `GeofenceResult` MAVLink message.

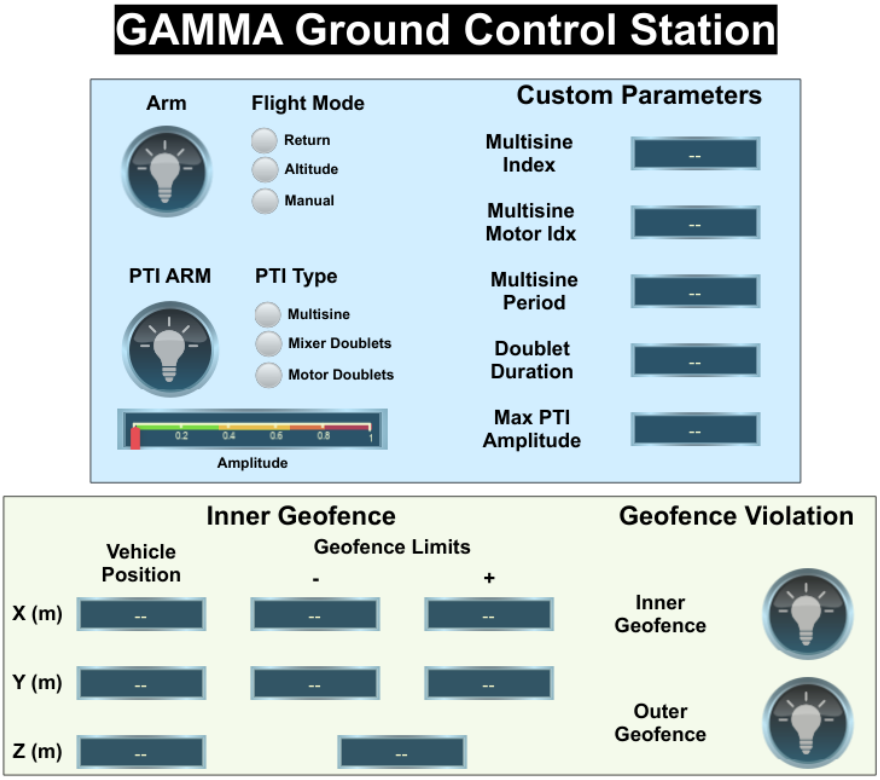


Figure 9: Custom ground station in Simulink®.

5 Flight Testing

This section outlines the flight testing of GAMMA accomplished to date. GAMMA was initially tested at the Autonomy Lab for Intelligent Flight Technology (ALIFT), an indoor facility at NASA Langley Research Center designed for UAS flight testing, and then at the City Environment Range Testing for Autonomous Integrated Navigation (CERTAIN), an outdoor flight range suitable for both fixed-wing and multi-rotor UAS [41]. Photos from initial flight tests are shown in Figure 10.

Figure 10: GAMMA flying at NASA Langley ALIFT facility (left) and CERTAIN range (right).

Flight testing at ALIFT was conducted to test the initial performance of the vehicle, tune inner loop control gains, and validate the implementation of PTIs. Various types of PTIs with different configuration parameters were tested. Figure 11 shows flight data of the vehicle during a 12.7-second multisine PTI with an amplitude of 12%. The period results in a frequency resolution of 0.5 rad/s and minimum identifiable frequency (second harmonic) of 1 rad/s, which were selected based on preliminary models of the vehicle dynamics in a hover. The dashed vertical lines indicate the beginning and end of the maneuver. The PTIs excite the vehicle motion in all degrees of freedom. The angular velocity and vertical acceleration data all exhibit high signal-to-noise ratios, which is an indicator of high-quality data for modeling. Figure 12 shows flight data of the vehicle during a series of mixer doublets, which excite the aircraft in one degree of freedom at a time. Each doublet has a duration of one second and an amplitude of 9%. During future flight tests, these types of maneuvers will be flown at various flight conditions to collect data for system identification.

Initial flight testing at CERTAIN was conducted to investigate the performance of automated flight modes, including altitude hold, position hold, and waypoint following, which were unsuitable for indoor testing. Additionally, these tests assessed the vehicle's forward flight performance. A rectangular flight plan was flown in the PX4 "Mission" flight mode, which is a fully automated waypoint-following mode. The ground track of the flight and waypoints are shown in Figure 13a. The flight path was repeated at ground speeds from 10 to 25 kts to collect performance data of the vehicle in forward flight at various speeds. Figure 13b shows airspeed, pitch angle, RPM, and current data collected during one leg of the flight. Airspeed was estimated by combining onboard ground speed measurements with wind measurements from a Vaisala WXT530 Series weather station. The average wind was 4 kts at 280° over the duration of the flight. Current measurements from both the Myxa ESC and the ACS724 are shown in the figure. The in-flight current data is much noisier than the bench test data, and the noise level for both sensors is comparable.

These preliminary flight test results were the culminating data from a 10-week summer internship project. In the future, this type of data will be used to measure vehicle performance at different steady-state flight conditions, which will be used for model development and comparisons with other data sources.

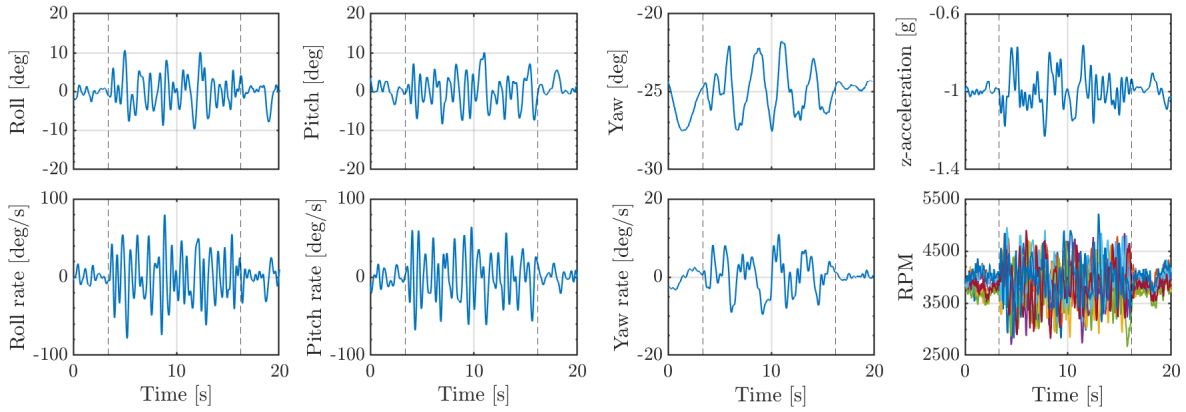


Figure 11: GAMMA response to multisine PTIs.

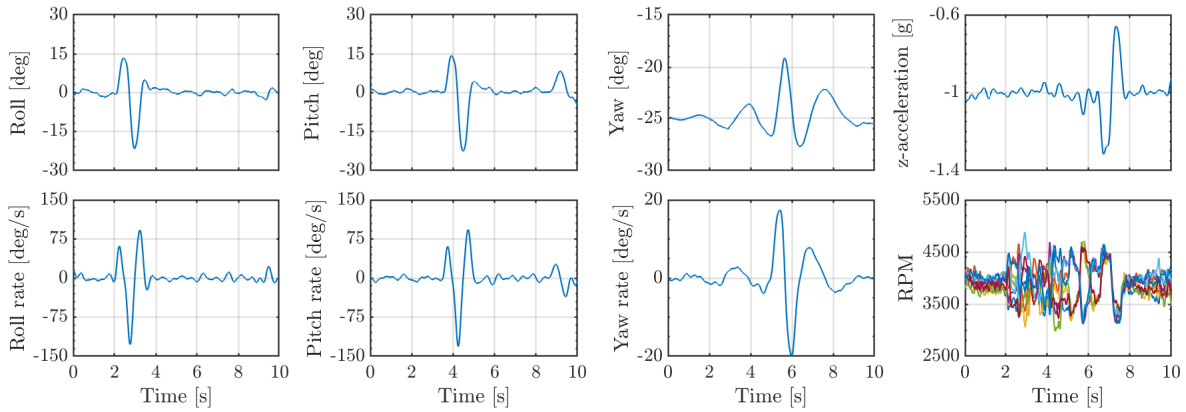
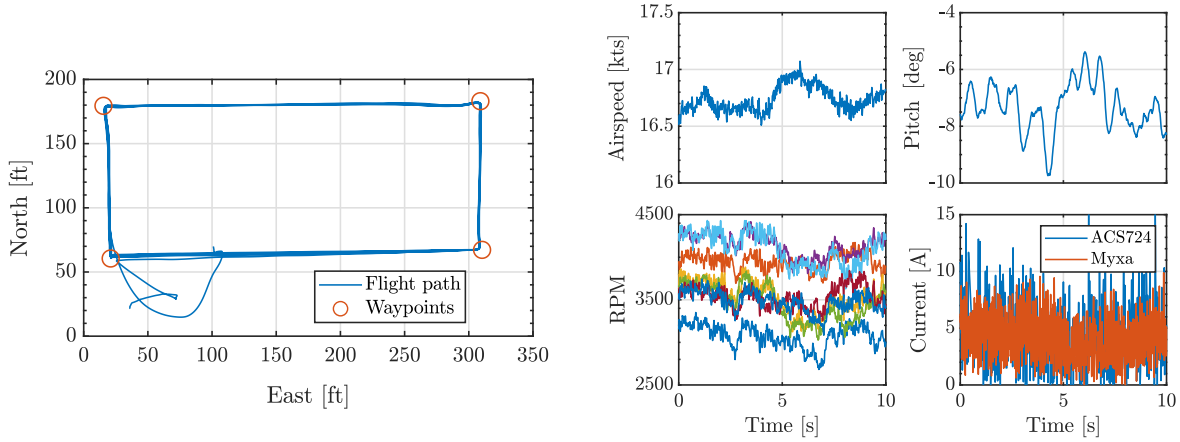


Figure 12: GAMMA response to mixer doublet PTIs.



(a) Ground track of CERTAIN range flight.

(b) Example steady state forward flight data.

Figure 13: GAMMA flight test data.

6 Conclusions and Future Work

This report detailed the design, development, and testing of the GAMMA vehicle, a platform developed to enhance multirotor aerodynamic modeling through efficient and effective flight testing. The GAMMA vehicle was equipped with avionics that were selected to accurately measure motor speed and power consumption, which are critical for generating reliable performance and flight dynamics models.

A new approach was developed to inject programmed test inputs to individual ESCs while maintaining the standard PX4 control architecture. This was achieved through the development of maneuver injection software added to PX4 using the UAV Toolbox. By integrating PTI capabilities with the standard PX4 control system, this approach improved the efficiency and safety of system identification flight testing, as custom control logic did not need to be designed, programmed, and tested. Moreover, additional capabilities were developed that contributed to increased safety and efficiency, such as the creation of the parameters for configuring PTI signal characteristics and the implementation of extra failsafe and geofence logic. Initial flight tests confirmed the ability of the platform to deliver dependable data for aerodynamic and power consumption modeling, highlighting its potential for further research.

Future work will focus on enhancing the current PTI framework to provide greater flexibility and control over the aircraft's behavior during flight. Further flight testing enabled by this research is planned, which will focus on flying system identification maneuvers at various steady-state flight conditions. This flight data will be used to develop aerodynamic and power consumption models and to analyze the vehicle dynamics over the full flight envelope. Finally, flight test results will also be compared to findings from wind tunnel tests.

Acknowledgments

This research was funded by the NASA Aeronautics Research Mission Directorate (ARMD) System Wide Safety (SWS) and Transformational Tools and Technologies (TTT) projects. The authors gratefully acknowledge the hardware development and testing support provided by George Altamirano, Matt Gray, Anthony Comer, Ben Simmons, Mario Smith, Matt Coldsnow, Dave Raney, Tom Ivanco, Nick Rymer, Brian Duvall, and others at NASA Langley Research Center.

References

1. Young, S. D., Ancel, E., Dill, E. T., Moore, A., Quach, C. C., Smalling, K. M., and Ellis, K. K., "Flight Testing In-Time Safety Assurance Technologies for UAS Operations," *AIAA AVIATION 2022 Forum*, 2022. https://doi.org/10.2514/6.2022-3458.

2. Ancel, E., Young, S. D., Quach, C. C., Haq, R. F., Darafsheh, K., Smalling, K. M., Vazquez, S. L., Dill, E. T., Condotta, R. C., Ethridge, B. E., Teska, L. R., and Johnson, T. A., "Design and Testing of an Approach to Automated In-Flight Safety Risk Management for sUAS Operations," *AIAA AVIATION 2022 Forum*, 2022. https://doi.org/10.2514/6.2022-3459.

3. Moore, A. J., Young, S. D., Altamirano, G., Ancel, E., Darafsheh, K., Dill, E., Foster, J., Quach, C., Mackenzie, A., Matt, J., Nguyen, T., Revolinksy, P., Smalling, K., Vazquez, S., Martin, L., Feldman, J., Gujral, V., Spirkovska, L., Hooey, B., Bradner, K., Iverson, D., Wolfe, S., Oza, N., Condotta, R., Morris, C., Glover, J. S., Rymer, N., Turner, A., Walter, C., Banerjee, P., Corbetta, M., Kulkarni, C., Cleland-Huang, J., Jones, J., and Bonin, T., "Testing of Advanced Capabilities to Enable In-time Safety Management and Assurance for Future Flight Operations," NASA TM–20230018665, 2024.

4. Ancel, E., Capristan, F. M., Foster, J. V., and Condotta, R. C., "In-Time Non-Participant Casualty Risk Assessment to Support Onboard Decision Making for Autonomous Unmanned Aircraft," *AIAA Aviation 2019 Forum*, 2019. https://doi.org/10.2514/6.2019-3053.

5. Foster, J. V., and Hartman, D., "High-Fidelity Multi-Rotor Unmanned Aircraft System (UAS) Simulation Development for Trajectory Prediction Under Off-Nominal Flight Dynamics," *17th AIAA Aviation Technology, Integration, and Operations Conference*, 2017. https://doi.org/10.2514/6.2017-3271.

6. Altamirano, G. V., Matt, J. J., Busan, R. C., and Foster, J. V., "Wind Tunnel Testing of Static Aerodynamic and Power Consumption Characteristics of an Octocopter," *AIAA AVIATION 2023 Forum*, 2023. https://doi.org/10.2514/6.2023-3967.

7. Altamirano, G. V., Matt, J. J., Farrell, S. M., and Palazzo, A., "Static and Forced Oscillation Wind Tunnel Tests of a Tandem Propeller Configuration," *AIAA SciTech 2025 Forum*, 2025. https://doi.org/https://doi.org/10.2514/6.2025-0664.

8. "Controller Diagrams," https://docs.px4.io/main/en/flight_stack/controller_diagrams.html, Accessed 01 August 2024.

9. Gresham, J. L., Fahmi, J.-M. W., Simmons, B. M., Hopwood, J. W., Foster, W., and Woolsey, C. A., "Flight Test Approach for Modeling and Control Law Validation for Unmanned Aircraft," *AIAA SCITECH 2022 Forum*, 2022. https://doi.org/10.2514/6.2022-2406.

10. Hopwood, J. W., and Woolsey, C. A., "Robust Linear Parameter-Varying Control for Safe and Effective Unstable Aircraft System Identification," *AIAA SCITECH 2024 Forum*, 2024. https://doi.org/10.2514/6.2024-1305.

11. Asper, G. D., Simmons, B. M., Axten, R. M., Ackerman, K. M., and Corrigan, P. E., "Inexpensive Multirotor Platform for Advanced Controls Testing (IMPACT): Development, Integration, and Experimentation," NASA TM–20240000223, 2024.

12. "PX4 Autopilot User Guide," https://docs.px4.io/master/en/, Accessed 01 August 2024.

13. "UAV Toolbox," https://www.mathworks.com/products/uav.html, Accessed 01 August 2024.

14. "HEX Here3+ CAN GNSS GPS Module," https://irlock.com/products/here3-gnss-m8p, Accessed 01 August 2024.

15. "CubePilot Cube Orange Flight Controller," *PX4 User Guide*, https://docs.px4.io/main/en/flight_controller/cubepilot_cube_orange.html, Accessed 01 August 2024.

16. "CC BEC Pro 20A Max Output," https://www.castlecreations.com/en/cc-bec-pro-010-0004-01, Accessed 01 August 2024.

17. "RFD900x Modem," https://rfdesign.com.au/rfd900x-modem/, Accessed 01 August 2024.

18. "DSMX Remote Receiver," https://www.spektrumrc.com/product/dsmx-remote-receiver/SPM9645.html, Accessed 01 August 2024.

19. "Arduino Mega 2560 Rev3," https://store.arduino.cc/products/arduino-mega-2560-rev3, Accessed 01 August 2024.

20. "Current Sensor Carrier 0A to 30A," https://www.pololu.com/product/4046, Accessed 01 August 2024.

21. "DEV-13743," https://www.digikey.com/en/products/detail/sparkfun-electronics/DEV-13743/5881845, Accessed 01 August 2024.

22. "Phoenix Edge Lite 50 Amp ESC," https://www.castlecreations.com/en/phoenix-edge-lite-50-esc-010-0113-00, Accessed 01 August 2024.

23. "AmpDrive AD0505 "Myxa"," https://zubax.com/products/drives/ampdrive/ad0505, Accessed 01 August 2024.

24. "TDK-Lambda Z100-2-U - Programmable DC Power Supply," https://www.tequipment.net/TDK-Lambda/Z100-2-U/DC-Power-Supplies/Lab-Power-Supplies/, Accessed 01 August 2024.

25. "Software Download for Genesys," https://product.tdk.com/en/products/power/switching-power/prg-power/designtool/software_vcp.html, Accessed 01 August 2024.

26. "FreqMesaureMulti Library," https://github.com/PaulStoffregen/FreqMeasureMulti, Accessed 01 August 2024.

27. "UAV Toolbox Support Package for PX4 Autopilots," https://www.mathworks.com/help/supportpkg/px4/, Accessed 01 August 2024.

28. Asper, G. D., and Simmons, B. M., "Rapid Flight Control Law Deployment and Testing Framework for Subscale VTOL Aircraft," NASA TM–20220011570, 2022.

29. "uORB Message Reference," *PX4 User Guide*, https://docs.px4.io/main/en/msg_docs/, Accessed 01 August 2024.

30. "MAVROS - ROS Wiki," https://wiki.ros.org/mavros, Accessed 01 August 2024.

31. "MAVLink Protocol Documentation," https://mavlink.io/en/, Accessed 01 August 2024.

32. Morelli, E. A., and Klein, V., *Aircraft System Identification: Theory and Practice*, 2nd ed., Sunflyte Enterprises, Williamsburg, VA, 2016.

33. Comer, A. M., Simmons, B. M., and Asper, G. D., "Design, Simulation, and Flight Testing of a Multi-Purpose VTOL Flight Control System," NASA TM (to appear), 2025.

34. Simmons, B. M., Gresham, J. L., and Woolsey, C. A., "Flight-Test System Identification Techniques and Applications for Small, Low-Cost, Fixed-Wing Aircraft," *Journal of Aircraft*, Vol. 60, No. 5, 2023, pp. 1503–1521. https://doi.org/10.2514/1.C037260.

35. Matt, J. J., "Comparison of Multisine Peak Factor Minimization Algorithms for Aircraft System Identification," NASA TM–20240009677, 2024.

36. "PX4 PWM Output Block Documentation," https://www.mathworks.com/help/uav/px4/ref/px4pwmoutput.html, Accessed 6 September 2024.

37. "PX4 Autopilot User Guide," https://docs.px4.io/master/en/, Accessed 01 August 2024.

38. "Plan View - GeoFence," *PX4 User Guide*, https://docs.qgroundcontrol.com/Stable_V4.3/zh/qgc-user-guide/plan_view/plan_geofence.html, Accessed 01 August 2024.

39. "VehicleCommand (UORB message)," *PX4 User Guide*, https://docs.px4.io/main/en/msg_docs/VehicleCommand.htmll, Accessed 01 August 2024.

40. "MAVLink Messaging," *PX4 User Guide*, https://docs.px4.io/main/en/middleware/mavlink.html, Accessed 01 August 2024.

41. Brown, J., "CERTAIN: City Environment Range Testing for Autonomous Integrated Navigation," NASA NF1676L-24405, 2016.

42. "Teensy Wiring Diagram," https://www.pjrc.com/teensy/pinout.html, Accessed 01 August 2024.

43. "Teensy onboard Real-Time Clock," https://www.adafruit.com/product/1870, Accessed 01 August 2024.

44. "Arduino IDE 2.2.1," https://www.arduino.cc/en/software, Accessed 01 August 2024.

45. "Package Teensy Index," https://www.pjrc.com/teensy/package_teensy_index.json, Accessed 01 August 2024.

46. "Tutorial 1: Software Setup," https://www.pjrc.com/teensy/tutorial.html, Accessed 01 August 2024.

47. "SD Library," https://www.arduino.cc/reference/en/libraries/sd/, Accessed 01 August 2024.

48. "DS1307RTC Library," https://www.arduino.cc/reference/en/libraries/ds1307rtc/, Accessed 01 August 2024.

49. "Castle Link V3 USB Programming Kit," https://www.castlecreations.com/en/castle-link-v3-usb-programming-kit-011-0119-00, Accessed 01 August 2024.

50. "Castle Link Software," https://home.castlecreations.com/download-castle-link/, Accessed 01 August 2024.

Appendix A

Vehicle Configuration

A custom power distribution board (PDB) was made to drive the motors and avionics shown in Figure A1. The connector from a 20 Ahr Li-Po battery was spliced in two directions, one powering a CubePilot Power Brick BEC and the other directed towards the PDB. The battery voltage was reduced through the CubePilot BEC to provide the Pixhawk Cube Orange+ flight controller with 5V. The PDB outputs nine power and ground cables, eight directed to the ESCs and one towards a Castle Creations BEC programmed to reduce the voltage required to meet the Arduino Mega 2560 power specifications. Although a ground cable runs directly to each motor, the power cables pass through a current sensor before continuing onto the motors. A full list of avionics used through this project are presented in Table 1.

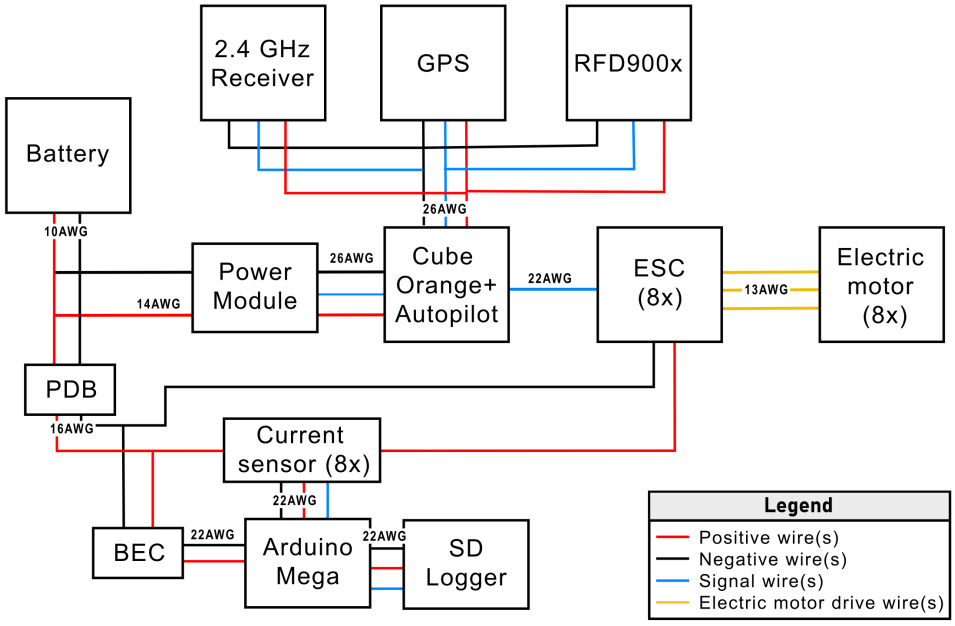


Figure A1: GAMMA wiring diagram.

Appendix B

Offboard Data Logging

B.1 Current

A Pololu ACS724 current sensor is connected to the power cables spanning from the power distribution board to the ESC to log the current of each motor. The current sensors receive 5V power and send a signal to an Arduino Mega 2530 analog port. An additional analog pin received a signal from the Cube Orange+ indicating the arm/disarm command to provide a basis for time syncing the data. The Arduino logs the current data received and puts the boolean arm signal onto an SD card using a Sparkfun Level Shifting microSD Breakout. The current data received provides accurate, current measurements independent of the data logged onboard the ESCs that are used for calibrating, comparing, and confirming onboard ESC current data.

The Arduino IDE software is used to enable current data reading and SD card logging on the Arduino. The current logging firmware uses the SD library to incorporate the data onto the SD card. The initialization parameters identify the allocation of the pins pertaining to the SD card, arm/disarm signal, and current data, as seen in Table B1.

Table B1: Arduino pin connections.

Arduino Pin	Board Connection	Pin Connection
Digital 50	SD Reader	DO
Digital 51	SD Reader	DI
Digital 52	SD Reader	SCK
Digital 53	SD Reader	CS
PWM 10	SD Reader	CD
Analog 0 - 7	Current Sensor	OUT
Analog 8	Cube Orange+	AUX1

The current sensor has a 5 V operating voltage, 0.5 V zero point, and sensitivity of 133 mv/A. A digital value for each current sensor is measured at 100 Hz using the `analogRead` function, which uses a 10-bit analog-to-digital converter. The digital value is converted to current using Equation B1

$$I = \frac{\frac{5000 * SV}{1023} - 500}{133} \tag{B1}$$

where I is the current in amps and SV is the sensor value using the `analogRead` function from the current sensor. A new text file is saved each time the Arduino is powered on. The index of the text file is allocated using the `findNextFileIndex` function, and the data file is established using the `SD.open` function from the SD library.

For accurate dynamic current modeling, it is essential to time-sync the current data and identify the delay in command and response of the current flow. Because both the Cube and Arduino logging have different startup times, the data was aligned through the commanded arm signal using the Cube's AUX rail. Figure B1 displays a diagram of writing the arm signal to the Arduino using a wire connecting the AUX1 port on the Cube to an Analog port on the Arduino. After receiving a boolean value from the ActuatorArmed uORB topic, the number is converted to a uint16 with a

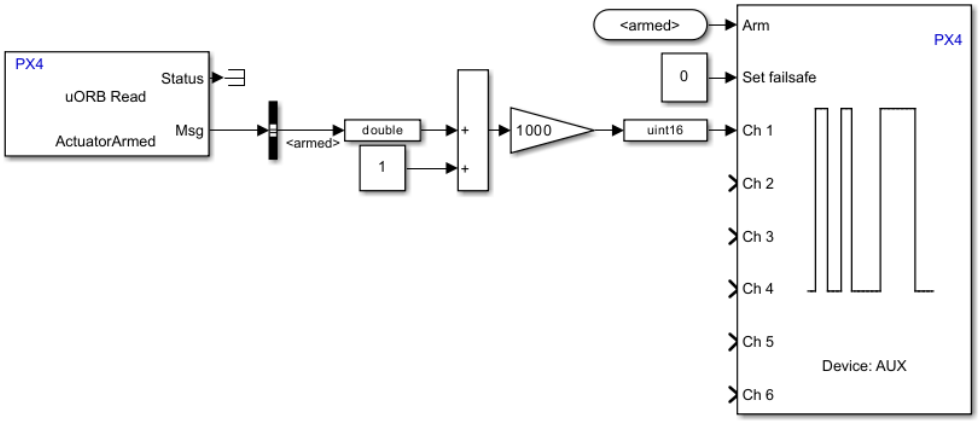


Figure B1: Command arm signal to Cube Orange+ AUX rail.

value of either 1000 or 2000 to be transmitted as a PWM signal and written to Channel 1 of the Cube Orange+ AUX rail. The Arduino received this signal and processed it using the `pulseIn` function.

B.2 RPM

To log the RPM of each motor, a PJRC Teensy 4.1 Development Board is integrated during the ESC comparison phase further outlined in Sec. 3 to log RPM signals from the Castle Creation Phoenix Edge series ESC auxiliary cable [42]. The data is logged from the motors individually onto a SanDisk Extreme 32GB MicroSD.

Firstly, the Teensy board was configured based on the diagram presented in Figure B2.

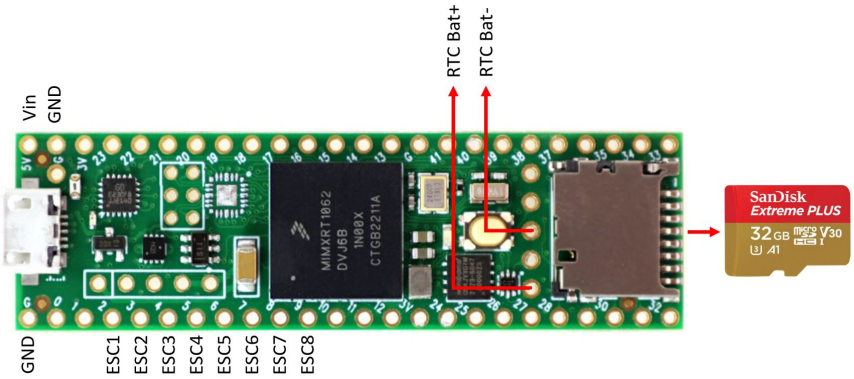


Figure B2: Teensy board wiring diagram derived from [42].

The Teensy has the capacity to measure output variables from up to 8 PWM signals corresponding to pins 2 through 9. In addition, a CR2032 Coin Battery Holder, acting as the onboard real-time clock, was soldered onto Teensy pins RTC Bat +, and RTC Bat − [43], power was supplied through pins Vin and GND, and the MicroSD card was placed onto the Teensy's onboard MicroSD card slot. The custom RPM logging Teensy firmware was installed onto the board using the Arduino IDE software [44] and the package Teensy index [45]. Following the installation of Arduino IDE and the package Teensy index, the steps published by PJRC were followed to upload

the RPM logging firmware onto the Teensy board [46].

The RPM logging firmware requires the installation of three libraries before running: SD library, DS1307RTC library, and FreqMeasureMulti library [26]. The SD library enables the reading and writing of firmware onto the SD card [47], the DS1307RTC library integrates the Teensy onboard real-time clock with the ESC RPM measurements [48], and the FreqMeasureMulti library measures the frequency of the wave that corresponds to the electrical RPM (eRPM) [26]. The limitation set by the FreqMeasureMulti library allows the user to record up to 8 RPM feedback signals from the Castle ESC auxiliary wire.

Following the installation of the libraries, the RPM logging firmware allocates a Teensy pin to a particular ESC. Five parameters are defined in the code, as described in Table B2. The firmware examined and recorded the number of pole pairs in each motor.

Table B2: RPM logging code parameters.

Variable	Description
N_ESCS	Number of ESCs to record from
N_MOTOR_POLES	Number of pole pairs in motor used
SAMPLING_RATE	Sampling rate for motor RPM
VEHICLE_NAME_TITLE	Vehicle name to be printed on outputting files
VERSION	Set of software versioning parameters

After the user configurable parameters are established and following a variety of setup and initialization functions, the code incorporates a loop that 1) reads PWM frequency from the auxiliary wire and 2) converts RPM to write data points to an SD card at a selected sampling rate. The Castle ESC auxiliary wire transmits RPM data by outputting a pulse for every electrical commutation of the motor. As previously stated, the FreqMeasureMulti library measures the frequency of the commutation signal, f, in Hertz, which is then converted into the mechanical RPM, which is the actual rotational velocity of the motor. For one motor revolution, the number of electric commutations is equal to the number of motor pole pairs (or half the total number of motor poles). Thus, the mechanical RPM is computed using Equation B2.

$$RPM = (f * 60)/(N\_MOTOR\_POLES/2) \tag{B2}$$

The resulting frequency and RPM vectors are saved as a .csv file containing a column of numbered data points spaced by an inputted time step in microseconds, followed by the RPM values for each of the four ESCs.

Once the Teensy board is uploaded with the required RPM logging firmware, each Castle ESC must be programmed using the Castle Link V3 USB programming kit [49] and the Castle Link software [50]. The Castle ESCs include a programmable auxiliary white wire that allows users to choose from a variety of data signals to record, including gain input, audible beacon, RPM out, and arm lock key.

REPORT DOCUMENTATION PAGE

Form Approved
OMB No. 0704–0188

1. REPORT DATE *(DD-MM-YYYY)*	2. REPORT TYPE	3. DATES COVERED *(From - To)*
01-03-2025	Technical Memorandum	

4. TITLE AND SUBTITLE
Design and Testing of an Octocopter for Aerodynamic and Power Consumption Modeling

5a. CONTRACT NUMBER

5b. GRANT NUMBER

5c. PROGRAM ELEMENT NUMBER

6. AUTHOR(S)
Patrick E. Corrigan
Office of STEM Engagement (OSTEM) Internship Program, Hampton, Virginia

Justin J. Matt and Garrett D. Asper
Langley Research Center, Hampton, Virginia

5d. PROJECT NUMBER

5e. TASK NUMBER

5f. WORK UNIT NUMBER
340428.02.40.07.01

7. PERFORMING ORGANIZATION NAME(S) AND ADDRESS(ES)
NASA Langley Research Center
Hampton, Virginia 23681-2199

8. PERFORMING ORGANIZATION REPORT NUMBER

9. SPONSORING/MONITORING AGENCY NAME(S) AND ADDRESS(ES)
National Aeronautics and Space Administration
Washington, DC 20546-0001

10. SPONSOR/MONITOR'S ACRONYM(S)
NASA

11. SPONSOR/MONITOR'S REPORT NUMBER(S)
NASA/TM–20240013453

12. DISTRIBUTION/AVAILABILITY STATEMENT
Unclassified-Unlimited
Subject Category
Availability: NASA STI Program (757) 864-9658

13. SUPPLEMENTARY NOTES

14. ABSTRACT
This report presents the design, development, and testing of the Generic Aerodynamic Modeling for Multirotor Aircraft (GAMMA) vehicle, a multirotor aircraft designed to collect flight data for model identification. An avionics suite was selected based on findings from bench tests to ensure that motor speed and power consumption can be measured accurately in flight. A custom maneuver injection software was developed for GAMMA that creates the capability to perform automated flight test maneuvers. This software was implemented using the Mathworks® UAV Toolbox and maintains the original control system of the PX4 open-source autopilot, eliminating the need to develop custom control logic for this capability. Moreover, additional functionalities were added to the software to improve flight test efficiency and safety, such as the creation of parameters for configuring flight maneuvers and the implementation of custom failsafe logic. Preliminary flight test results are presented which demonstrate the effectiveness of the developed approach for collecting flight data for performance and flight dynamics modeling.

15. SUBJECT TERMS
flight testing, flight controls, system identification, PX4, Pixhawk, UAV, GAMMA

16. SECURITY CLASSIFICATION OF:			17. LIMITATION OF ABSTRACT	18. NUMBER OF PAGES	19a. NAME OF RESPONSIBLE PERSON
a. REPORT	b. ABSTRACT	c. THIS PAGE			HQ-STI-infodesk@mail.nasa.gov
U	U	U	UU	25	19b. TELEPHONE NUMBER *(Include area code)* (757) 864-9658